Rule Engine Design: Handling `numOfSegments`

📌 Overview

In our Rule Engine (Drools-based), we needed to handle pricing logic that depends on the number of flight segments (`numOfSegments`). Two different approaches were implemented and benchmarked:

1. **Approach A: Handle `numOfSegments` inside the `then` block**
2. **Approach B: Split rules based on `numOfSegments` into separate rules**

This document explains both approaches, their implementation strategy, and the performance impact observed during profiling.

⚙️ Approach A — Conditional Logic in `then` Block

💡 Idea

Instead of creating multiple rules for each segment case, we keep a **single rule** and handle the logic using `if-else` inside the `then` block.

🧩 Example Concept

```
if ($RulesCommonBean.getNumOfSegments() == 1) {
    // apply pricing logic for 1 segment
} else if ($RulesCommonBean.getNumOfSegments() == 2) {
    // apply pricing logic for 2 segments
} else {
    // default logic
}
```

✅ Advantages

- ✔️ Fewer rules → simpler rule base
- ✔️ Easier to maintain and modify

- ✔️ Reduces rule duplication
- ✔️ Keeps conditions centralized

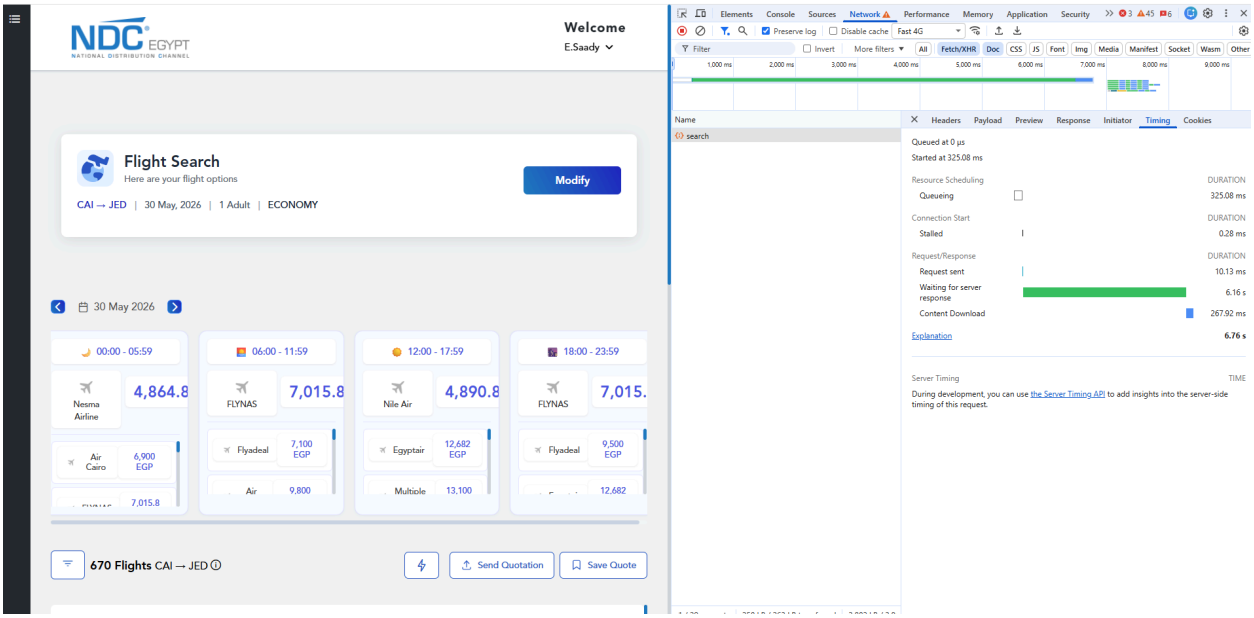
❌ Disadvantages

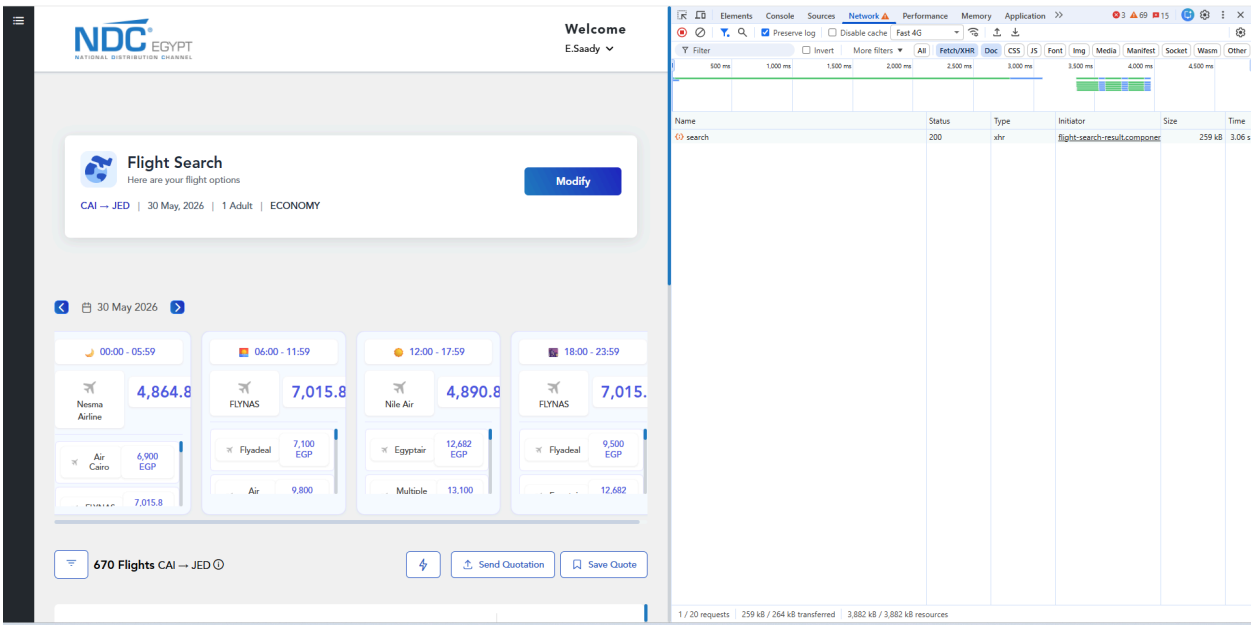
- ❌ Slightly more logic inside `then`
- ❌ Rule becomes less declarative (more imperative logic)

Drool File will be like

```
rule "TestFareBasisCode_1715"
 when
        $RulesCommonBean:RulesCommonBean( ruleType == "discOnSc" &&  ticketingPeriod >= "10-Apr-2026 00:00" && ticketingPeriod < "01-Jan-2030 00:00"  && new HashSet(Arrays.asList( "67", "2")).contains(countryId) && new
HashSet(Arrays.asList( "11476", "11446", "11444")) not contains (branchId) &&  null != (agencyId))
$RulesFlightBean:RulesFlightBean((new HashSet(Arrays.asList( "CAI")).contains(originAirport)))

 then
        if ($RulesCommonBean.getNumOfSegments() == 1) {$RulesCommonBean.setServiceChargePrice($RulesCommonBean.getServiceChargePrice() + 600);} else if ($RulesCommonBean.getNumOfSegments() == 2)
{$RulesCommonBean.setServiceChargePrice($RulesCommonBean.getServiceChargePrice() + 1000);} else {$RulesCommonBean.setServiceChargePrice($RulesCommonBean.getServiceChargePrice() +
500.0);}$RulesCommonBean.getServiceChargeAdultList().add($RulesCommonBean.getServiceChargePrice());$RulesCommonBean.setServiceChargePriceChild(0);$RulesCommonBean.setServiceChargePriceChild($RulesCommonBean.getServiceChargePriceChild()+
500.0);$RulesCommonBean.getServiceChargeChildList().add($RulesCommonBean.getServiceChargePriceChild());$RulesCommonBean.setServiceChargePriceInfant(0);
$RulesCommonBean.setServiceChargePriceInfant($RulesCommonBean.getServiceChargePriceInfant()+500.0);$RulesCommonBean.getServiceChargeInfantList().add($RulesCommonBean.getServiceChargePriceInfant());
 end
```





⚙️ Approach B — Splitting Rules by `numOfSegments`

💡 Idea

Create **separate rules**, each with a specific condition for `numOfSegments`.

🧩 Example Concept

```
rule "flightType_1716"
when
    numOfSegments == 0 (implicitly or general case)
then
    // pricing logic
end

rule "flightType_1716_seg1"
when
    numOfSegments == 1
then
    // different pricing logic
end
```

✅ Advantages

- ✔️ More declarative and readable rules
- ✔️ Each rule is focused and isolated

- ✔️ Aligns well with Drools pattern matching philosophy

❌ Disadvantages

- ❌ Rule duplication increases
- ❌ Harder to maintain when logic grows
- ❌ Larger rule base → slower rule evaluation
- ❌ More conditions to evaluate during matching phase

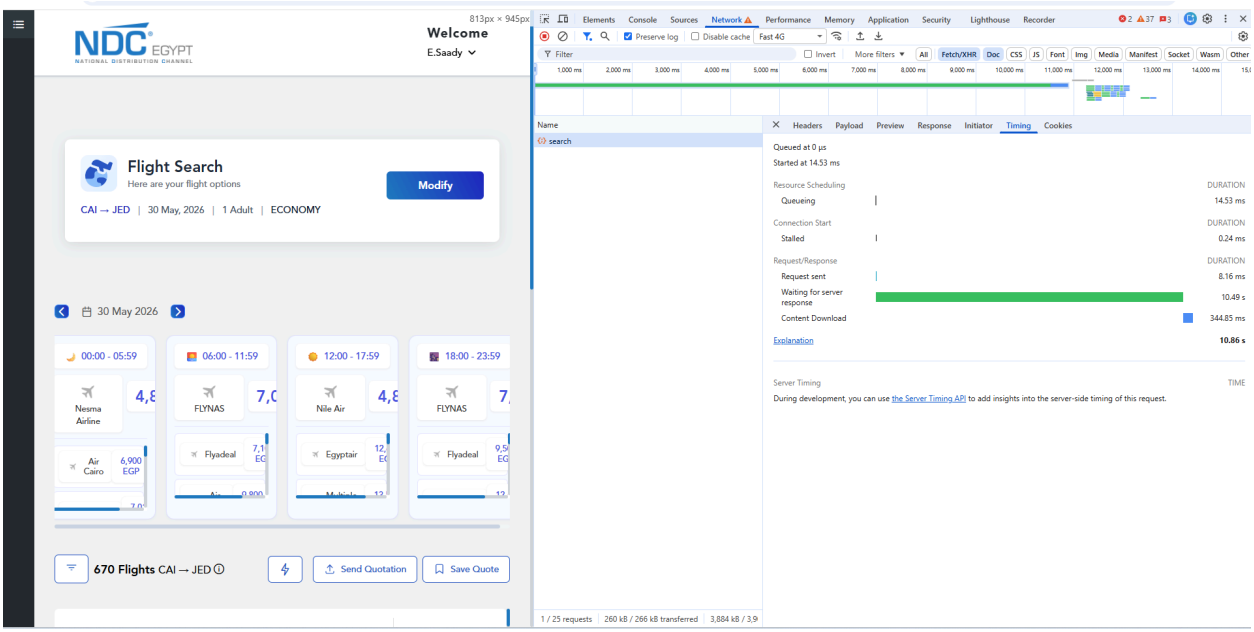
Drool File Will be

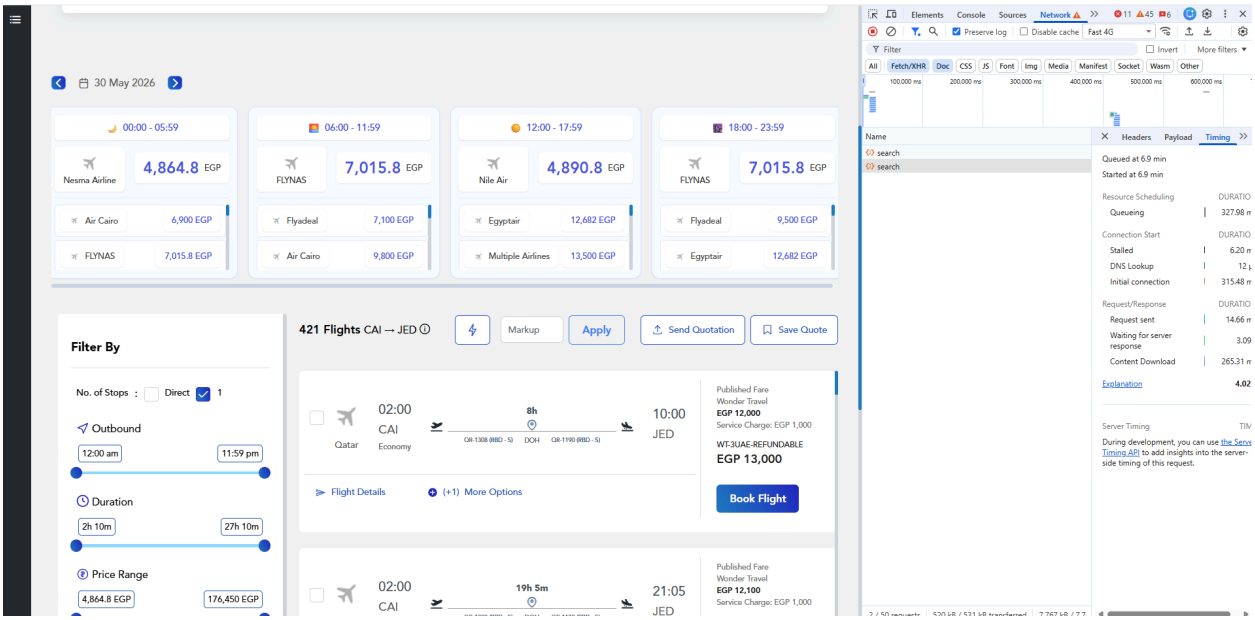
```
rule "flightType_1716"
 when
        $RulesCommonBean:RulesCommonBean( ruleType == "discOnSc" &&  ticketingPeriod >= "21-Apr-2026 00:00" && ticketingPeriod < "31-May-2028 00:00"  && new HashSet(Arrays.asList( "67", "2")).contains(countryId) &&  null == (agencyId)
&& new HashSet(Arrays.asList( "948")).contains(branchId))
$RulesFlightBean:RulesFlightBean(((flightType == "DIRECT")))

 then
        $RulesCommonBean.setDiscountPrice(0);$RulesCommonBean.setDiscountPrice($RulesCommonBean.getDiscountPrice()+100.0);$RulesCommonBean.getDiscountAdultList().add($RulesCommonBean.getDiscountPrice());
$RulesCommonBean.getDiscIdAdult().put($RulesCommonBean.getDiscountPrice(),1716);$RulesCommonBean.setDiscountPriceChild(0);$RulesCommonBean.setDiscountPriceChild($RulesCommonBean.getDiscountPriceChild()+100.0);
$RulesCommonBean.getDiscountChildList().add($RulesCommonBean.getDiscountPriceChild());$RulesCommonBean.getDiscIdChild().put($RulesCommonBean.getDiscountPriceChild(),1716);$RulesCommonBean.setDiscountPriceInfant(0);
$RulesCommonBean.setDiscountPriceInfant($RulesCommonBean.getDiscountPriceInfant()+100.0);$RulesCommonBean.getDiscountInfantList().add($RulesCommonBean.getDiscountPriceInfant());
$RulesCommonBean.getDiscIdInfant().put($RulesCommonBean.getDiscountPriceInfant(),1716);
 end


 rule "flightType_1716_seg1"
 when
        $RulesCommonBean:RulesCommonBean( ruleType == "discOnSc" &&  ticketingPeriod >= "21-Apr-2026 00:00" && ticketingPeriod < "31-May-2028 00:00"  && new HashSet(Arrays.asList( "67", "2")).contains(countryId) &&  null == (agencyId)
&& new HashSet(Arrays.asList( "948")).contains(branchId))
$RulesFlightBean:RulesFlightBean(((flightType == "DIRECT")) && numOfSegments == 1)

 then
        $RulesCommonBean.setDiscountPrice(0);$RulesCommonBean.setDiscountPrice($RulesCommonBean.getDiscountPrice()+200.0);$RulesCommonBean.getDiscountAdultList().add($RulesCommonBean.getDiscountPrice());
$RulesCommonBean.getDiscIdAdult().put($RulesCommonBean.getDiscountPrice(),1716);$RulesCommonBean.setDiscountPriceChild(0);$RulesCommonBean.setDiscountPriceChild($RulesCommonBean.getDiscountPriceChild()+200.0);
$RulesCommonBean.getDiscountChildList().add($RulesCommonBean.getDiscountPriceChild());$RulesCommonBean.getDiscIdChild().put($RulesCommonBean.getDiscountPriceChild(),1716);$RulesCommonBean.setDiscountPriceInfant(0);
$RulesCommonBean.setDiscountPriceInfant($RulesCommonBean.getDiscountPriceInfant()+200.0);$RulesCommonBean.getDiscountInfantList().add($RulesCommonBean.getDiscountPriceInfant());
$RulesCommonBean.getDiscIdInfant().put($RulesCommonBean.getDiscountPriceInfant(),1716);
 end
```





⚙️ Approach C — Set `numOfSegments` map in RulesCommonBean

💡 Idea

Create a map inside RulesCommonBean to add the whole numOfSegments in each rule and its prices.

rule_1 → {segm_1 → 500.0}

rule_2 → {segm_1 → 600.0}

🧩 Example Concept

```
 then
  $RulesCommonBean.setServiceChargePrice(0); $RulesCommonBean.setServiceChargePrice($Rule
sCommonBean.getServiceChargePrice() + $RulesCommonBean.getPriceMap().get(1715).get($Rules
CommonBean.getNumOfSegments()));$RulesCommonBean.getServiceChargeAdultList().add($RulesCo
mmonBean.getServiceChargePrice());$RulesCommonBean.setServiceChargePriceChild(0);$RulesCo
mmonBean.setServiceChargePriceChild($RulesCommonBean.getServiceChargePriceChild()+500.
0);$RulesCommonBean.getServiceChargeChildList().add($RulesCommonBean.getServiceChargePric
eChild());$RulesCommonBean.setServiceChargePriceInfant(0);$RulesCommonBean.setServiceChar
gePriceInfant($RulesCommonBean.getServiceChargePriceInfant()+500.0);$RulesCommonBean.getS
```

```
erviceChargeInfantList().add($RulesCommonBean.getServiceChargePriceInfant());
 end
```

✅ Advantages

- You do not need to add each segment and each price inside Drool file
- You just fetch the whole Rules' segments once

❌ Disadvantages

- Need to update this map after specific period of time

Drool File Will be



```
rule "TestFareBasisCode_1715"
when
    $RulesCommonBean:RulesCommonBean( ruleType == "discOrSc" &&  ticketingPeriod >= "09-Apr-2026 00:00" && ticketingPeriod < "31-Dec-2029 00:00"  && new HashSet(Arrays.asList( "67", "2")).contains(countryId) && new HashSet(Arrays.asList( "11476", "11446",
"11444")) not contains (branchId) &&  null != (agencyId))
$RulesFlightBean:RulesFlightBean((new HashSet(Arrays.asList( "CAI")).contains(originAirport)))

then
    $RulesCommonBean.setServiceChargePrice(0): $RulesCommonBean.setServiceChargePrice($RulesCommonBean.getServiceChargePrice() +
$RulesCommonBean.getPriceMap().get(1715).get($RulesCommonBean.getNumOfSegments())):$RulesCommonBean.getServiceChargeAdultList().add($RulesCommonBean.getServiceChargePrice()):$RulesCommonBean.setServiceChargePriceChild(0):$RulesCommonBean.s
etServiceChargePriceChild($RulesCommonBean.getServiceChargePriceChild()+500.0):$RulesCommonBean.getServiceChargeChildList().add($RulesCommonBean.getServiceChargePriceChild()):$RulesCommonBean.setServiceChargePriceInfant(0):$RulesCommonBean.set
ServiceChargePriceInfant($RulesCommonBean.getServiceChargePriceInfant()+500.0):$RulesCommonBean.getServiceChargeInfantList().add($RulesCommonBean.getServiceChargePriceInfant());
end
```

⚡ Performance Profiling Results

Test Scenario

- Flight search execution
- First server run (cold start)
- Same dataset and rule set

📈 Analysis

🔍 Why Approach A is Faster

- Fewer rules → less pattern matching overhead
- Drools evaluates fewer `when` conditions
- Logic executed after matching, not during matching

🔍 Why Approach B is Slower

- Increased number of rules → more comparisons in working memory
- Each rule adds overhead in the **Rete network**
- Pattern matching becomes heavier

Key Insight

> In Drools, **conditions in the `when` clause are more expensive than logic in the `then` clause**.

So:

- Moving logic into `then` reduces matching complexity
- Splitting rules increases the size of the rule network

📊 Results

Approach	Description	Execution Time 1st search hit	Execution Time after 1st search hit
Approach A	Condition inside `then`	**~8 seconds**	**~4 seconds**
Approach B	Split rules by segments	**~12 seconds**	**~5 seconds**
Approach C	Use a global map to map each segment with each specific price	**~7 seconds**	**~3 seconds**